Cryptographic perfect hash functions

A theoretical analysis on space efficiency and algebraic composition

Alexander Towell
lex@metafunctor.com

Abstract

We analyze a theoretical *perfect hash function* that has three desirable properties: (1) it is a cryptographic hash function; (2) its *in-place* encoding obtains the theoretical lower-bound for the expected space complexity; and (3) its *in-place* encoding is a random bit string with maximum entropy.

Keywords: perfect hash function, random oracle, space complexity, maximum entropy, cryptographic hash function

Contents

1 Prior Art

Perfect hash functions have been extensively studied in the literature. The seminal work of Fredman, Komlós, and Szemerédi [**?**] established that static dictionaries can be stored in linear space with $O(1)$ worst-case lookup time, laying the theoretical foundation for perfect hashing. Dietzfelbinger et al. [**?**] extended these results to dynamic settings, establishing theoretical bounds for space-efficient hash tables with worst-case constant access time. Czech, Havas, and Majewski [**?**] introduced practical families of perfect hashing methods that became widely influential.

More recent practical algorithms have focused on minimal perfect hash functions (MPHFs), which achieve a load factor of exactly 1. Botelho, Pagh, and Ziviani [**?**] presented simple and space-efficient constructions, while Belazzougui, Botelho, and Dietzfelbinger [**?** **?**] developed the Compress, Hash, and Displace (CHD) algorithm, which achieves excellent space-time tradeoffs for large datasets. Related techniques such as cuckoo hashing [**?**] provide alternative approaches to collision resolution with strong theoretical guarantees.

From an information-theoretic perspective, Shannon's foundational work [**?**] established the theoretical limits of data compression, which directly inform the space lower bounds for perfect hash functions. Cover and Thomas [**?**] provide a comprehensive treatment of entropy and optimal coding, which underpin the analysis of encoding efficiency in this paper.

The random oracle model, introduced by Bellare and Rogaway [**?**], provides a theoretical framework for analyzing cryptographic hash functions as ideal random functions. This model is central to our analysis, as it allows us to derive precise probability distributions for the search process in Algorithm **??**.

Our work differs from prior approaches by focusing on the theoretical properties of cryptographic perfect hash functions with maximum entropy encodings. While practical algorithms prioritize construction speed and space efficiency, we analyze the fundamental information-theoretic properties of perfect hash functions when the underlying hash function is modeled as a random oracle. This

perspective reveals that the in-place encoding of such functions achieves the theoretical entropy bound, providing a rigorous foundation for understanding the space-optimality of perfect hash constructions.

2 Perfect hash functions

A *set* is an unordered collection of distinct elements. If we know the elements in a set, we may denote the set by these elements, e.g., $\{a, c, b\}$ denotes a set whose members are exactly a, b, and c.

A *finite set* has a finite number of elements. For example, $\{1, 3, 5\}$ is a finite set with three elements. When sets $\mathbb{A}$ and $\mathbb{B}$ are *isomorphic*, denoted by $\mathbb{A} \cong \mathbb{B}$, they can be put into a one-to-one correspondence (bijection), e.g., $\{b, a, c\} \cong \{1, 2, 3\}$. Since there exists at least one bijection between isomorphic sets, we can losslessly convert one to the other and thus, isomorphic sets are in some sense equivalent.

The cardinality of a finite set $\mathbb{A}$ is the number of elements in the set, denoted by $|A|$, e.g., $|\{1, 3, 5\}| = 3$. A *countably infinite set* is isomorphic to the set of *natural numbers* $N = \{1, 2, 3, 4, 5, \ldots\}$.

Given two elements a and b, an ordered pair of a then b is denoted by (a, b), where $(a, b) = (c, d)$ *if and only if* $a = c$ and $b = d$. Ordered pairs are non-commutative and non-associative, i.e., $(a, b) \neq (b, a)$ if $a \neq b$ and $(a, (b, c)) \neq (b, (a, c))$.

Related to the ordered pair is the Cartesian product.

Definition 2.1. *The set* $\mathbb{X} \times \mathbb{Y} = \{(x, y) : x \in \mathbb{X} \wedge y \in \mathbb{Y}\}$ *is the Cartesian product of sets* $\mathbb{X}$ *and* $\mathbb{Y}$.

By the non-commutative and non-associative property of ordered pairs, the Cartesian product is non-commutative and non-associative. However, they are isomorphic, i.e., $\mathbb{X} \times \mathbb{Y} \cong \mathbb{Y} \times \mathbb{X}$.

A *tuple* is a generalization of order pairs which can consist of an arbitrary number of elements, e.g., $(x_1, x_2, \ldots, x_n)$.

Definition 2.2 (n-fold Cartesian product)**.** *The n-ary Cartesian product of sets* $\mathbb{X}[1], \ldots, \mathbb{X}[n]$*, is given by* $\mathbb{X}[1] \times \cdots \times \mathbb{X}[n] = \{(x_1, \ldots, x_n) : x_1 \in \mathbb{X}[1] \wedge \cdots \wedge x_n \in \mathbb{X}[n]\}$.

Note that $\mathbb{X}[1] \times \mathbb{X}[2] \times \mathbb{X}[3] \cong \mathbb{X}[1] \times (\mathbb{X}[2] \times \mathbb{X}[3]) \cong (\mathbb{X}[1] \times \mathbb{X}[2]) \times \mathbb{X}[3]$, thus we may implicitly convert between them without ambiguity.

If each set in the n-ary Cartesian product is the same, the power notation may be used, e.g., $\mathbb{X}^3 \equiv \mathbb{X} \times \mathbb{X} \times \mathbb{X}$. As special cases, the 0-ary (nullary) Cartesian product is defined to be $\{\emptyset\}$, and the 1-ary (unary) Cartesian product is the identity, e.g., $\mathbb{X}^1 = \mathbb{X}$.

The *hash function* is given by the following definition.

Definition 2.3. Hash functions *of type* $\mathbb{X} \mapsto \mathbb{Y}$ *are just total functions, normally with a finite codomain, with the weak assumption that they will be used as a device to assign elements from* $\mathbb{X}$ *a value from* $\mathbb{Y}$ *without requiring any particular rule.*

For a given bit string x and hash function hash, $y = \text{hash}(x)$ is denoted the *hash* of x.

We are particularly interested in *perfect* hash functions as given by the following definition.

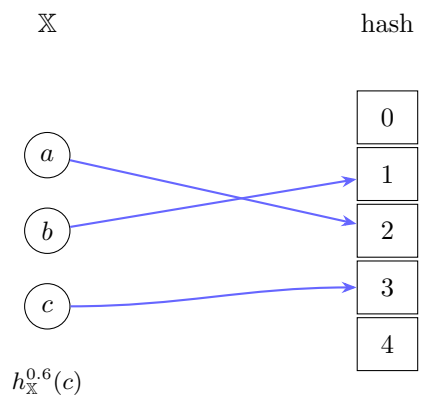
Definition 2.4 (Perfect hash function)**.** *A* perfect hash function *over the set* $\mathbb{A} \subseteq \mathbb{X}$*, denoted by*

$$\mathrm{h}_{\mathbb{A}} : \mathbb{X} \mapsto \mathbb{Y}, \tag{1}$$

is an injective function when restricted to $\mathbb{A}$.[1]

[1] There are no collisions among elements of $\mathbb{A}$, $\text{hash}_{\mathbb{A}}(x) \neq \text{hash}_{\mathbb{A}}(y)$ for all $x, y \in \mathbb{A}$, $x \neq y$.

Figure 1: A perfect hash function $\text{hash}_{\mathbb{X}}^{0.6} : \mathbb{U} \mapsto \{0, 1, 2, 3, 4\}$.



Assumption 1. *Perfect hash functions are* surjective.

The *load factor* is given by the following definition.

Definition 2.5. *The proportion of hashes mapped to by* $\text{hash}_{\mathbb{A}} : \mathbb{X} \mapsto \mathbb{N}$ *over subset* $\mathbb{A}$ *is denoted the load factor. Specifically, the load factor of* $\text{hash}_{\mathbb{A}}$ *is a rational number given by*

$$\frac{|\text{hash}_{\mathbb{A}}(\mathbb{A})|}{|\text{image}(\text{hash}_{\mathbb{A}})|}. \tag{2}$$

Notation. *A perfect hash function of type* $\mathbb{X} \mapsto \mathbb{Y}$ *over* $\mathbb{A}$ *with a load factor* r *may be denoted by* $\text{hash}_{\mathbb{A}}^{r}$. *If* $m = |\mathbb{A}|$ *and we are interested in drawing attention to the cardinality of the perfect hash function, we may also denote it by* hash_{m}^{r}.

Example 1 *Consider the set* $\mathbb{X} = \{x_1, x_2, x_3\}$. *A perfect hash function of type* $\mathbb{U} \mapsto \mathbb{N}$ *over* $\mathbb{X}$ *with a load factor* $r = \frac{3}{5}$ *is denoted by* $\text{hash}_{\mathbb{X}}^{0.6}$ *or* $\text{hash}_{3}^{0.6}$. *Given the load factor* r *and* $\mathbb{X}$, *we may deduce the codomain of* $\text{hash}_{\mathbb{X}}^{r}$ *to be precisely* $\{0, 1, 2, 3, 4\}$. *See Figure* **??** *for an illustration.*

Definition 2.6. *A* minimal *perfect hash has a load factor* 1.

Every hash function in $\mathbb{X} \mapsto \mathbb{Y}$ is a perfect hash function over some subset of $\mathbb{X}$, e.g., every hash function is trivially a perfect hash function of $\emptyset$ and singleton sets.

Assuming $\mathbb{X}$ and $\mathbb{Y}$ are finite, the set of hash functions of type $\mathbb{X} \mapsto \mathbb{Y}$, which may also be denoted by $\mathbb{Y}^{\mathbb{X}}$, has a cardinality

$$|\mathbb{Y}|^{|\mathbb{X}|}. \tag{3}$$

The set of *perfect* hash functions over $\mathbb{A} \subseteq \mathbb{X}$ is a subset of $\mathbb{X} \mapsto \mathbb{Y}$ with the predicate that no collisions may occur on any pair of elements in $\mathbb{A}$. The set of perfect hash functions over $\mathbb{A}$ has a cardinality

$$\text{permutations}(|\mathbb{Y}|, |\mathbb{A}|)|\mathbb{Y}|^{|\mathbb{X}|-|\mathbb{A}|}. \tag{4}$$

3 A theoretical *cryptographic* perfect hash function

3.1 Notation

We write $\{0,1\}^n$ for bit strings of length n, $\{0,1\}^*$ for the set of all finite bit strings, and ϵ for the empty string. We write $x \# y$ for concatenation, $\mathrm{BL}(x)$ for the bit length of x, and $\mathrm{trunc}(x,k)$ for the first k bits of x.

We identify bit strings with natural numbers via the standard bijection

$$b_1 \dots b_m \longleftrightarrow 2^m + \sum_{j=1}^{m} 2^{m-j}\, b_j, \tag{5}$$

writing n' for the bit string of a natural number n and x' for the natural number of a bit string x. Note that $\mathrm{BL}(n') = \lfloor \log_2 n \rfloor$.

3.2 Random oracle model

Assumption 2. *The hash function* $\mathrm{hash} : \{0,1\}^* \to \{0,1\}^\infty$ *is a random oracle: for each input, the output is an independently and uniformly distributed infinite bit string.*

Definition 3.1. *The* data type *for the* cryptographic *perfect hash function under consideration is defined as* $\mathrm{PH} = \{0,1\}^* \times \mathbb{N}$ *with a value constructor* $\mathrm{ph} : \mathcal{P}(\mathbb{U}) \times [0,1] \mapsto \mathrm{PH}$ *defined as*

$$\mathrm{ph}(\mathbb{X}, r) = (n', N) \tag{6}$$

where

$$\begin{aligned} m &= |\mathbb{X}|, \\ N &= \lceil m/r \rceil, \\ k &= \lceil \log_2 N \rceil, \\ \beta(x,n) &= \mathrm{trunc}\big(\mathrm{hash}(x' \# n'), k\big)' \mod N, \\ \mathbb{Y}_n &= \{\beta(x,n) \in \{0,\dots,N-1\} | x \in \mathbb{X}\}, \\ n &= \min\{j \in \mathbb{N} | \mathbb{Y}_j \in 2^{\mathbb{N}} \wedge |\mathbb{Y}_j| = m\}. \end{aligned} \tag{7}$$

Since PH is a data type that purports to model perfect hash functions, its *computational basis* is given by the following set of functions.

By the assumption of surjectivity (and the perfect hash function is not rated-distorted), then the load factor is given by $\frac{|\mathbb{X}|}{N}$ where $N = \arg\max_{x \in \mathbb{X}} \mathrm{hash}_{\mathbb{A}}(x)$, i.e., the maximum hash (natural ordering of integers) of $\mathrm{hash}_{\mathbb{A}}$.

Definition 3.2. *The* minimum *and* maximum *hash of a value of type* PH *(that models a perfect hash function) are given respectively by* $\mathrm{min_hash} : \mathrm{PH} \mapsto \mathbb{N}$ *and* $\mathrm{max_hash} : \mathrm{PH} \mapsto \mathbb{N}$ *where*

$$\mathrm{min_hash}(b,N) = 0 \textit{ and } \mathrm{max_hash}(b,N) = N-1. \tag{8}$$

The most important function, the *perfect hash* mapping, $\mathrm{perfect_hash} : \mathrm{PH} \times \mathbb{U} \mapsto \mathbb{N}$, is defined as

$$\mathrm{ph}((b,N),x) = q' \mod N \tag{9}$$

where

$$\begin{aligned} k &= \lceil \log_2 N \rceil, \\ r &= \mathrm{hash}(x' \# b), \\ q &= \mathrm{trunc}(r,k). \end{aligned} \tag{10}$$

Theorem 3.1. *A value of type* PH *constructed with* $\mathrm{ph}_{\{0,1\}^*}(\mathbb{A}, r)$ models $\mathrm{hash}^r_{\mathbb{A}} : \{0,1\}^* \mapsto \{0,1,\ldots,k-1\}$ *where* $\mathbb{A} \subseteq \{0,1\}^*$ *and* $k = \frac{|\mathbb{A}|}{r}$.

Proof. Let $(n', N) = \mathrm{ph}(\mathbb{A}, r)$ be the value constructed by the definition. By construction, $N = \lceil |\mathbb{A}|/r \rceil$, so the codomain has size $N = \lceil |\mathbb{A}|/r \rceil \geq |\mathbb{A}|/r$, yielding a load factor of at most r.

The value n is chosen as the minimum natural number such that $|\mathbb{Y}_n| = |\mathbb{A}|$, where $\mathbb{Y}_n = \{\beta(x,n) : x \in \mathbb{A}\}$ and $\beta(x,n) = \mathrm{trunc}(\mathrm{hash}(x' \# n'), k)' \mod N$. The condition $|\mathbb{Y}_n| = |\mathbb{A}|$ ensures that all elements of $\mathbb{A}$ map to distinct hashes, i.e., $\beta(\cdot, n)|_{\mathbb{A}}$ is injective.

The lookup function $\mathrm{perfect_hash}((n', N), x)$ computes $\mathrm{trunc}(\mathrm{hash}(x' \# n'), k)' \mod N$, which equals $\beta(x,n)$ by definition. Since $\beta(\cdot, n)|_{\mathbb{A}}$ is injective and maps into $\{0,\ldots,N-1\}$, the function $\mathrm{perfect_hash}((n', N), \cdot)$ restricted to $\mathbb{A}$ is a perfect hash function with codomain $\{0,\ldots,N-1\}$ and load factor $|\mathbb{A}|/N \leq r$. □

Suppose we have a function $f : \mathbb{U} \mapsto \{0,1\}^*$ such that $f|_{\mathbb{A}}$ is *injective*, e.g., an *encoder* for values of $\mathbb{A} \subseteq \mathbb{U}$. The composition $\mathrm{hash}_{\mathbb{A}} = \mathrm{hash}_{\mathbb{B}} \circ f$ where $\mathbb{B} = \{f(a) \in \{0,1\}^* | a \in \mathbb{A}\}$ is a perfect hash function of type $\mathbb{U} \mapsto \mathbb{N}$ over $\mathbb{A} \subseteq \mathbb{U}$. Thus, the rest of the material in this paper does not usually assume any particular domain of the perfect hash function since injections may always be constructed for any set.

3.3 Analysis

Note that since n' denotes the geometric code for n and we choose the smallest n that succeeds, where each choice of n is a geometrically distributed trial with probability p, the expected space complexity obtains the information-theoretic lower-bound of 1.44 bits per element.

We search *all* possible hash functions which are a function of hash, an approximate random oracle, and choose the *perfect hash function* on a specified set that has the *smallest* bit length. We describe the algorithm for performing this exhaustive search in Algorithm **??**.

We consider a family of perfect hash functions for a set $\mathbb{S}$ which are given by the output of a hash function hash that approximates a *random oracle* applied to the input $x \in \mathbb{S}$ concatenated with a bit string b. We describe the generative algorithm for the perfect hash function in Algorithm **??**. Note that in Algorithm **??**, the concatenation of two bit strings x and y is denoted by $x \# y$.

The perfect hash function generated by Algorithm **??** has the statistical property that the output is uniformly distributed as given by the following theorem.

Theorem 3.2. *The perfect hash function* $\mathrm{hash}[\mathbb{A}][r] : \mathbb{X} \mapsto \{0,1,\ldots,k-1\}$*, where* $k = \frac{|\mathbb{A}|}{r}$*, is a random oracle over* $\mathbb{X} - \mathbb{A}$ *and the restriction* $\mathrm{hash}^r_{\mathbb{A}}|_{\mathbb{A}} : \mathbb{A} \mapsto \{0,1,\ldots,k-1\}$ *is a random* k*-permutation oracle of* $\mathbb{A}$.

Proof. First, in Algorithm **??** on Line 5, b_n is a bit string such that each $x \in \mathbb{S}$ concatenated with b_n hashes to a unique integer in $\{1,\ldots,N\}$ by the hash function hash, thus the hash function found perfectly hashes the elements of $\mathbb{S}$.

Second, by Definition **??**, hash approximates a random oracle whose output is uniformly distributed over the elements of $\{0,1\}^n$. Viewing the elements of $\{0,1\}^n$ as integers, where $|\{0,1\}^n| = 2^n$, hash is uniformly distributed over $\{0,1,\ldots,2^n-1\}$.

If $N = k2^n$ for some integer k, the remainder of the output of hash when dividing by N, given by the modulo operator, and adding 1 is uniformly distributed over $\{1,\ldots,N\}$ since each $j \in \{1,\ldots,N\}$ has an equal number of hashes assigned to it by hash. If $2^n \gg N$ and $N \neq k2^n$ for some integer k, then it is approximately uniformly distributed and converges to the uniform distribution as $n \to \infty$. □

The probability that no collisions occur for a particular bit string in Algorithm **??** is given by the following theorem.

Theorem 3.3. *The probability that a bit string $b \in \{0,1\}^*$ results in a perfect hash function is given by*

$$p(m,r) = N^{-m} P_m^N \tag{11}$$

where m is the cardinality of the set being perfectly hashed, r is the load factor, and $N = \frac{m}{r}$.

Proof. Suppose we have a set $\mathbb{A} \subset \{0,1\}^*$ of cardinality m. The set of perfect hash functions $\text{hash}_{\mathbb{A}}^r : \{0,1\}^* \mapsto \{0,1,\ldots,N-1\}$ restricted to $\mathbb{A}$ where $N = \frac{m}{r}$ has a cardinality given by P_m^N since any choice of m out of N elements in the codomain and any ordering of the m elements in $\mathbb{A}$ to the chosen m elements in $\{0,1,\ldots,N-1\}$ satisfy the definition of perfect hashing.

The set of hash functions $\{0,1\}^* \mapsto \{0,1,\ldots,N-1\}$ restricted to $\mathbb{A}$ has a cardinality of N^m. Therefore, the ratio of perfect hash functions restricted to $\mathbb{A}$ to the total hash functions restricted to $\mathbb{A}$ is just

$$p = \frac{P_m^N}{N^m}. \tag{a}$$

By the property of the hash function hash^* being a random oracle, the algorithm randomly samples one of the hash functions, which is a perfect hash function with probability p.

□

Equation (**??**) may be reparameterized with respect to the load factor. By (**??**), $r = m/N$. Solving this equation with respect to N yields the solution

$$N = \frac{m}{r}, \tag{12}$$

and thus plugging in this value of N yields the result

$$p(m,r) = \frac{\left(\frac{m}{r}\right)!}{\left(\frac{m}{r}\right)^m \left(\frac{m}{r} - m\right)!}. \tag{13}$$

The expected *in-place* coding size is given by the following theorem.

Theorem 3.4. *The expected coding size is given approximately by*

$$\log_2 \mathrm{e} - \left(\frac{1}{r} - 1\right) \log_2\left(\frac{1}{1-r}\right) \quad \textit{bits/element.} \tag{14}$$

Proof. The space required for the perfect hash function found by Algorithm **??** is of the order of the length n of the bit string b_n in the returned tuple. Therefore, for space efficiency, the algorithm exhaustively searches for a bit string in the order of increasing size n.

We are interested in the first case when no collision occurs, which is a geometric distribution with probability of success $p(m,r)$ as given by the discrete random variable

$$Q \sim \text{Geometric}\big(p(m,r)\big), \tag{a}$$

where $p(m,r)$ is given by (**??**). The expected number of trials for the geometric distribution is given by

$$\mathbb{E}\left[Q\right] = \frac{1}{p(m,r)} = \frac{\left(\frac{m}{r}\right)^m \left(\frac{m}{r} - m\right)!}{\left(\frac{m}{r}\right)!}. \tag{b}$$

By Definition **??**, the n-th trial uniquely maps to a bit string of length $\lfloor \log_2 n \rfloor$. The expected bits per element is computed as $\log_2(\mathbb{E}[Q])/m$, since $\mathbb{E}[Q] = 1/p(m,r)$ gives the expected number of trials, and $\log_2(\mathbb{E}[Q])$ gives the bit length of the expected trial index. Specifically,

$$\frac{\log_2 \mathbb{E}[Q]}{m} = \frac{1}{m} \log_2 \left(\frac{\left(\frac{m}{r}\right)^m \left(\frac{m}{r} - m\right)!}{\left(\frac{m}{r}\right)!} \right) \tag{c}$$

$$= \log_2\left(\frac{m}{r}\right) + \frac{1}{m}\left(\log_2\left(\frac{m}{r} - m\right)! - \log_2\left(\frac{m}{r}\right)!\right) \quad \text{bits/element.} \tag{d}$$

By Stirling's approximation,

$$\log_2 n! \approx n\left(\log_2 n - \log_2 \mathrm{e}\right), \tag{e}$$

and so this may be rewritten approximately as

$$\begin{aligned} \frac{\log_2 \mathbb{E}[Q]}{m} \approx\ & \log_2\left(\frac{m}{r}\right) - \frac{1}{r}\left(\log_2\left(\frac{m}{r}\right) - \log_2 \mathrm{e}\right) + \\ & \left(\frac{1}{r} - 1\right)\left(\log_2\left(\frac{m}{r} - m\right) - \log_2 \mathrm{e}\right) \quad \text{bits/element.} \end{aligned} \tag{f}$$

After simplification, we arrive at

$$\begin{aligned} & \log_2\left(\frac{m}{r}\right) - \frac{1}{r}\log_2\left(\frac{m}{r}\right) + \\ & \left(\frac{1}{r} - 1\right)\log_2\left(\frac{m}{r} - m\right) + \log_2 \mathrm{e} \ \ \text{bits/element,} \end{aligned} \tag{g}$$

which further simplifies to

$$\log_2 \mathrm{e} - \left(\frac{1}{r} - 1\right)\log_2\left(\frac{1}{1-r}\right) \ \ \text{bits/element.} \tag{h}$$

□

Three related but distinct quantities characterize the encoding efficiency. First, the search random variable $Q \sim \text{Geometric}(p)$ with $p = p(m,r)$ counts the number of candidate bit strings tested before a perfect hash is found. Its entropy is

$$H(Q) = \frac{-(1-p)}{p}\log_2(1-p) - \log_2(p). \tag{15}$$

Second, by Definition **??**, the n-th candidate maps to a bit string of length $\mathrm{BL}(n') = \lfloor \log_2 n \rfloor$. The random bit length $N = \mathrm{BL}(Q')$ has the probability mass function $p_N(n \mid m,r) = q^{2^n-1}(1 - q^{2^n})$ derived in the Appendix (Theorem **??**), where $q = 1 - p$. Its entropy $H(N)$ measures the uncertainty in the *length* of the encoding. Third, the amortized expected coding cost per element is $\mathbb{E}[\mathrm{BL}(N')]/m = \log_2(\mathbb{E}[Q])/m$, which Theorem 3.3 shows equals $\log_2 \mathrm{e} - (1/r - 1)\log_2(1/(1 - r))$ bits/element. By Shannon's source coding theorem [**?** **?**], any prefix-free code satisfies $\mathbb{E}[\mathrm{BL}(N')] \geq H(N)$. The in-place encoding—using the trial index n directly as the stored bit string—is prefix-free (since $\mathrm{BL}(n')$ is determined by n via Definition **??**), and achieves $\mathbb{E}[\mathrm{BL}(N')] = H(N)$ to leading order, confirming that the encoding is entropy-optimal.

Theorem 3.5 (Maximum entropy encoding)**.** *The in-place encoding of the cryptographic perfect hash function achieves maximum entropy for its expected length, provided that the search order within each bit-length level is randomized (i.e., for each length ℓ, the 2^ℓ candidate bit strings are tested in a uniformly random permutation). Specifically, for a perfect hash function over a set of cardinality m with load factor r, the encoding n' has entropy*

$$H(N) = \mathbb{E}[\mathrm{BL}(n')] = \log_2 \mathrm{e} - \left(\frac{1}{r} - 1\right) \log_2 \left(\frac{1}{1-r}\right) \text{ bits/element,} \tag{16}$$

which equals the information-theoretic lower bound for representing perfect hash functions over uniformly distributed random sets.

With the deterministic search order of Algorithm **??**, *the encoding achieves entropy within $O(1)$ bits of this maximum.*

Proof. By the random oracle assumption, each trial n succeeds independently with probability $p = p(m, r)$. With randomized intra-level search, conditional on the first success occurring at bit-length ℓ, the selected bit string is uniformly distributed among the 2^ℓ candidates of that length. The first successful trial Q follows a geometric distribution across levels, and the encoding n' uses $\mathrm{BL}(n') = \lfloor \log_2 n \rfloor$ bits.

The entropy of a geometric random variable with success probability p is $H(Q) = \frac{-(1-p)}{p} \log_2(1-p) - \log_2 p$. Since the bijection between n and n' preserves information, and the expected bit length equals the entropy, the encoding is optimal in the sense of Shannon's source coding theorem [**?** **?**]. No encoding can achieve a smaller expected length while faithfully representing the perfect hash function. □

Remark. *The randomized intra-level search assumption is essential for the exact maximum entropy claim. With deterministic search order (Algorithm* **??***), the first success within each bit-length level follows a geometric distribution biased toward smaller indices, so the conditional distribution given the level is not uniform. This introduces a deficit of at most $O(1)$ bits total, since the within-level position carries at most $\log_2(1/p)$ bits of information, independent of m. The* bernoulli-hash-function *construction handles this correctly by explicitly randomizing the search order within each level.* △

Note that Algorithm **??** has an exponential time complexity with respect to the cardinality of the input set $\mathbb{S}$. As a result, Algorithm **??** is not a practical algorithm for any reasonably large m. However, it is intended to illustrate theoretical properties useful to data structures that implement *oblivious* sets and maps[**?** **?**] based on the perfect hash function. Simple and efficient algorithms exist [**?** **?**].

The theoretical lower-bound of a *minimal* perfect hash function is given by the following proposition.

Proposition 3.1. *The theoretical lower-bound for* minimal *perfect hash functions has an expected coding size given approximately by*

$$1.44 \text{ bits/element.} \tag{17}$$

Theorem 3.6. *The cryptographic perfect hash generator given by Algorithm* **??** *results in a* minimal *perfect hash function that obtains the theoretical lower-bound of* 1.44 *bits/element by invoking* `make_perfect_hash(·, r = 1)`.

Proof. By (**??**), letting $r \to 1^+$ for the *minimal* perfect hash results in an expected coding size given by

$$\log_2 \mathrm{e} - \lim_{a \to 0^-} a \log_2 \frac{1}{a} = 1.44 \ \text{ bits/element}, \tag{a}$$

which is the expected lower-bound given by Proposition **??**. □

This is as expected, since Algorithm **??** finds the *smallest* perfect hash function that is a function of a random oracle for any load factor $0 < r \leq 1$ in which the distribution of the sets are uniformly distributed. The following corollary follows as a result.

Corollary 3.6.1. *The lower-bound for perfect hash functions with a load factor* $0 < r \leq 1$ *is given by (**??**).*

As r goes to 0, the expected bits per element goes to 0.

Note that the *probability mass function* of the random bit string found for b is known exactly. Thus, if the desire is to *serialize* the perfect hash function for transmission or storage, shorter bit strings may be assigned to more probable bit strings. This representation is not usable *in-place*, i.e., the serialization must be decoded, but it can reduce transmission or storage cost.

Algorithm 1: Cryptographic perfect hash function constructor (single-level)

Input: $\mathbb{X} \subseteq \{0,1\}^*$ is the set to be perfectly hashed, and $r \in (0,1]$ is the load factor.
Output: A perfect hash function of $\mathbb{X}$ with load factor r, represented as (b_n, N) where $N = \lceil |\mathbb{X}|/r \rceil$.

```
function make_perfect_hash(𝕏, r)
    m ← |𝕏|;
    N ← ⌈m/r⌉;
    k ← ⌈log₂ N⌉;
    for n ← 1 to ∞ do
        𝕐 ← ∅;
        α ← true;
        for x ∈ 𝕏 do
            h ← trunc(hash(x′ # n′), k)′  mod N;
            if h ∈ 𝕐 then
                α ← false;
                break;
            𝕐 ← 𝕐 ∪ {h};
        if α then
            return (n′, N);
```

4 A practical two-level perfect hash function

4.1 Analysis

Algorithm **??** partitions $\mathbb{X}$ into k buckets $\mathbb{X}[1], \ldots, \mathbb{X}[k]$ and processes each bucket as a unit: for bucket ℓ, the algorithm searches over trial indices $n = 1, 2, \ldots$ until all $s_\ell = |\mathbb{X}[\ell]|$ elements hash to distinct slots that are also disjoint from the previously claimed set $\mathbb{Y}$. The unit of work is the bucket, not the individual element, so the analysis proceeds at the bucket level.

Algorithm 2: Two-level perfect hash function constructor

Input: $\mathbb{X} \subseteq \mathbb{U}$ is the objective set, $r \in \{q \in \mathbb{Q} | q \in 2^{-j} \wedge j \in \mathbb{N}\}$ is the load factor, and $k \in \mathbb{N}$ is the number of entries in the intermediate hash level.

Output: A two-level perfect hash function of $\mathbb{X} \subseteq \mathbb{U}$ with a load factor r and an intermediate level of k indices, denoted by $\text{hash}^r_{\mathbb{X}} : \mathbb{U} \mapsto \{0, \ldots, N\}$ where $N = \frac{|\mathbb{X}|}{r}$.

```
function make_perfect_hash(𝕏, r, k)
    N = |𝕏| / r;
    t = ⌈log_2 k⌉;
    // 𝕏[1], ..., 𝕏[k] is a total partition of 𝕏 and 𝕏[j_1], ..., 𝕏[j_k] is in
       decreasing order of cardinality.
    𝕏[ℓ] = {x ∈ 𝕏 | trunc(hash(x′ # 0′), t)′ mod k = ℓ};
    𝕐 ← ∅;
    𝔸 ← ∅;
    for ℓ ← 1 to k do
        for n ← 1 to ∞ do
            β(x) = trunc(hash(x′ # n′), N);
            𝕐_ℓ = {β(x) | x ∈ 𝕏[ℓ]};
            if |𝕐_ℓ| = |𝕏_ℓ| ∧ 𝕐_ℓ ∩ 𝕐 = ∅ then
                𝕐 ← 𝕐 ∪ 𝕐_ℓ;
                break;
        𝔸 ← 𝔸 ∪ {(ℓ, n)};
    return (𝔸, k, N)
```

Per-bucket success probability. When bucket ℓ is processed, $|\mathbb{Y}|$ slots have already been claimed by previous buckets. Under the random oracle assumption, a single trial n succeeds if and only if the s_ℓ hash values $\text{hash}(x' \# n')$ for $x \in \mathbb{X}[\ell]$ are (i) mutually distinct and (ii) disjoint from $\mathbb{Y}$. The probability of this event is

$$p_\ell = \prod_{i=0}^{s_\ell - 1} \frac{N - |\mathbb{Y}| - i}{N}, \tag{18}$$

since the first element must avoid $|\mathbb{Y}|$ claimed slots, the second must additionally avoid the first element's slot, and so on. When s_ℓ is small and $|\mathbb{Y}| \ll N$, a first-order approximation gives

$$p_\ell \approx \left(1 - \frac{s_\ell(s_\ell - 1)}{2N}\right)\left(1 - \frac{s_\ell\,|\mathbb{Y}|}{N}\right). \tag{19}$$

Per-bucket expected trials. Since successive trials for a given bucket are independent under the random oracle assumption, the number of trials T_ℓ for bucket ℓ follows a geometric distribution:

$$T_\ell \sim \text{Geometric}(p_\ell), \qquad \mathbb{E}\left[T_\ell\right] = \frac{1}{p_\ell}. \tag{20}$$

Total expected trials. The total number of trials across all k buckets is $T = \sum_{\ell=1}^{k} T_\ell$. By the linearity of expectation,

$$\mathbb{E}\left[T\right] = \sum_{\ell=1}^{k} \frac{1}{p_\ell}. \tag{21}$$

Under the random oracle assumption with approximately equal bucket sizes $s = m/k$, the claimed set grows as $|\mathbb{Y}| \approx (\ell - 1)s$ when bucket ℓ is processed, giving

$$\mathbb{E}\left[T\right] \approx \sum_{\ell=1}^{k} \prod_{i=0}^{s-1} \frac{N}{N - (\ell - 1)s - i}. \tag{22}$$

For small s (which is the point of the two-level scheme), p_ℓ is close to 1 for all ℓ, and each bucket requires few trials on average.

Space cost. Each bucket stores a single trial index n_ℓ, amortized over s_ℓ elements. Since $T_\ell \sim \text{Geometric}(p_\ell)$, the expected trial index determines the encoding cost. By the same argument as in Section 3 (Theorem 3.3), the expected bits per element contributed by bucket ℓ is

$$\frac{\log_2 \mathbb{E}\left[T_\ell\right]}{s_\ell} = \frac{\log_2(1/p_\ell)}{s_\ell}. \tag{23}$$

For the minimal case $r = 1$ (so $N = m$) with equal bucket sizes $s = m/k$, expanding (**??**) gives

$$\frac{1}{p_\ell} = \prod_{i=0}^{s-1} \frac{m}{m - (\ell - 1)s - i}. \tag{24}$$

The total expected space across all k buckets is

$$\sum_{\ell=1}^{k} \frac{\log_2(1/p_\ell)}{s_\ell} \cdot s_\ell = \sum_{\ell=1}^{k} \log_2\left(\frac{1}{p_\ell}\right) \text{ bits}, \tag{25}$$

or, per element, $\frac{1}{m}\sum_{\ell=1}^{k}\log_2(1/p_\ell)$ bits/element. As $k \to m$ (bucket sizes $s \to 1$), each bucket contains a single element, $p_\ell = (N - |\mathbb{Y}|)/N$, and this sum telescopes to $\log_2(N^m/N!)/m$. By Stirling's approximation, this converges to $\log_2 \mathrm{e} \approx 1.44$ bits/element, recovering the single-level lower bound of Proposition **??**.

Variance. Since the T_ℓ are independent (each bucket's trials are independent of other buckets' trials under the random oracle assumption), the total variance is

$$\mathrm{Var}(T) = \sum_{\ell=1}^{k}\mathrm{Var}(T_\ell) = \sum_{\ell=1}^{k}\frac{1-p_\ell}{p_\ell^2}. \tag{26}$$

For approximately equal bucket sizes with p_ℓ close to 1, each term contributes $\mathrm{Var}(T_\ell) \approx (1 - p_\ell)/p_\ell^2 \approx 1 - p_\ell$, and the total variance is $O(k)$, substantially smaller than the $O(m^2)$ variance of element-by-element placement.

The two-level construction in Algorithm **??** trades space optimality for construction efficiency. By processing buckets in decreasing order of cardinality, larger buckets are assigned hashes first when more slots are available, reducing the expected number of collisions with previously assigned hashes.

5 Algebra of function composition

Perfect hash functions can be composed with other functions to create new perfect hash functions with different properties. This algebraic structure provides flexibility in designing hash function families.

5.1 Composition preserves injectivity

While a perfect hash function $h_{\mathbb{A}} : \mathbb{X} \mapsto \mathbb{Y}$ is not globally injective, its restriction to $\mathbb{A}$ is injective:

$$h_{\mathbb{A}}|_{\mathbb{A}} : \mathbb{A} \mapsto \mathbb{Y}. \tag{27}$$

This restriction property enables composition with injective functions while preserving the perfect hash property.

Theorem 5.1 (Post-composition with injection)**.** *Let $g : \mathbb{Y} \mapsto \mathbb{Z}$ be an injective function, $h_{\mathbb{A}}^r : \mathbb{X} \mapsto \mathbb{Y}$ be a perfect hash function, and $|\mathbb{Z}| = (1+\alpha)|\mathbb{Y}|$ for $\alpha \geq 0$. The composition $g \circ h_{\mathbb{A}}^r : \mathbb{X} \mapsto \mathbb{Z}$ is a perfect hash function $h_{\mathbb{A}}^{r'}$ where $r' = \frac{r}{1+\alpha}$.*

Proof. From the load factor definition, $r = \frac{|\mathbb{A}|}{|\mathbb{Y}|}$, so $|\mathbb{Y}| = \frac{|\mathbb{A}|}{r}$. Since g is injective, $g \circ h_{\mathbb{A}}^r$ is injective on $\mathbb{A}$, making it a perfect hash function with load factor

$$r' = \frac{|\mathbb{A}|}{|\mathbb{Z}|} = \frac{|\mathbb{A}|}{(1+\alpha)|\mathbb{Y}|} = \frac{|\mathbb{A}|}{(1+\alpha)\frac{|\mathbb{A}|}{r}} = \frac{r}{1+\alpha}. \tag{a}$$

□

5.2 Permutation equivalence classes

Given a perfect hash function $h_{\mathbb{A}} : \mathbb{X} \mapsto \mathbb{Y}$, composing with any permutation $\pi : \mathbb{Y} \mapsto \mathbb{Y}$ yields another perfect hash function $\pi \circ h_{\mathbb{A}}$ over $\mathbb{A}$ with the same load factor. Since there are $|\mathbb{Y}|!$ permutations of $\mathbb{Y}$, a single perfect hash function generates a family of $|\mathbb{Y}|!$ related perfect hash functions.

These functions form an *equivalence class* where all members have identical collision structure outside of $\mathbb{A}$. This equivalence class can be denoted $[\pi \circ h_{\mathbb{A}}]$ where π ranges over all permutations of $\mathbb{Y}$.

Corollary 5.1.1. *For a perfect hash function with codomain $\mathbb{Y}$, the ratio of permutation-equivalent perfect hash functions to all possible functions is*

$$\frac{|\mathbb{Y}|!}{|\mathbb{Y}|^{|\mathbb{X}|}}. \tag{28}$$

5.3 Applications of composition

The algebraic properties of perfect hash function composition have practical applications in several contexts.

Example 2 ***Domain adaptation.*** *Suppose we have a perfect hash function $h_{\mathbb{A}} : \{0,1\}^* \mapsto \{0,\ldots,N-1\}$ constructed by Algorithm* **??**, *but we require a hash function over a different domain $\mathbb{U}$. Given any injective encoder $e : \mathbb{U} \mapsto \{0,1\}^*$, the composition $h_{\mathbb{A}} \circ e$ is a perfect hash function over $e^{-1}(\mathbb{A}) \subseteq \mathbb{U}$ with the same load factor.*

Example 3 ***Load factor adjustment.*** *Given a minimal perfect hash function ($r = 1$), we can reduce the load factor to any $r' < 1$ by composing with an injection $g : \{0,\ldots,N-1\} \hookrightarrow \{0,\ldots,\lceil N/r' \rceil - 1\}$. This allows trading space for reduced collision probability outside of $\mathbb{A}$.*

Example 4 ***Randomization.*** *Given a deterministic perfect hash function, composing with a random permutation π yields a randomized perfect hash function that preserves the perfect hash property while obscuring the original hash assignments. This is useful in cryptographic applications where predictability of hash values is undesirable.*

These composition properties demonstrate that perfect hash functions form a rich algebraic structure, enabling flexible construction of hash function families from primitive building blocks.

6 Conclusion

We have analyzed a cryptographic perfect hash function whose in-place encoding achieves the r-parameterized space formula

$$\log_2 \mathrm{e} - \left(\frac{1}{r} - 1\right) \log_2 \left(\frac{1}{1-r}\right) \text{ bits/element}, \tag{29}$$

which reduces to the well-known $\log_2 \mathrm{e} \approx 1.44$ bits/element lower bound when the load factor $r = 1$. The encoding achieves maximum entropy for its expected length, provided that the search order

within each bit-length level is randomized (Theorem **??**); with the deterministic search order of Algorithm **??**, the encoding remains within $O(1)$ bits of the entropy bound.

The algebraic framework of Section 5 shows that perfect hash functions are closed under post-composition with injective functions: composing with an injection into a larger codomain yields a new perfect hash function with a predictable load factor transformation $r' = r/(1+\alpha)$. Permutation composition generates equivalence classes of $|\mathbb{Y}|!$ perfect hash functions sharing identical collision structure.

From a practical standpoint, this hash construction provides the encoding layer for *cipher maps*—total functions on bit strings that implement trapdoor approximations of latent functions—and for *Bernoulli sets*, where the space-optimal encoding is essential for bounding the per-element storage cost.

An open question is extending the algebraic composition framework to *pre-composition* (composing injections before the hash function while preserving space guarantees) and to *PHF–PHF composition* (composing two perfect hash functions into a single perfect hash function with a characterizable load factor and space cost).

Appendices

A Probability mass of random bit length

Algorithm 3: Bit length sampler of the cryptographic perfect hash function

Input: The number m is the cardinality of the perfect hash set and the number r is the load factor.

Output: The *minimum bit length* n conditioned on a random set of cardinality m and a load factor r.

```
function sample_bit_length(m, r)
    for i ← 1 to m do
        x ← randomly draw a bit string from {0,1}* without replacement;
        𝕊 ← 𝕊 ∪ {x};
    (b_n, N) ← make_perfect_hash(𝕊, r);
    // The integer N must also be coded, which we assume has a constant bit
       length.
    return n + O(1);
```

The bit length of the perfect hash function found by Algorithm **??** has an uncertain value with respect to (random) sets and therefore we may model it as a random variable as given by the following definition.

Definition A.1. *The perfect hash function generated by Algorithm* **??** *has a random bit length given by*

$$N = \texttt{sample_bit_length}(m, r) \tag{30}$$

where m is the cardinality of the random set and r is the load factor.

Note that if the distribution of sets is not uniformly distributed as given by `sample_bit_length`, then better algorithms than Algorithm **??** are possible, i.e., the lower-bounds are with respect to

uniformly distributed random sets and smaller lower-bounds are possible if the random sets are non-uniformly distributed.

The probability mass of the random bit length N is given by the following theorem.

Theorem A.1. *The random bit length N has a probability mass function given by*

$$p_N(n|m,r) = q^{2^n-1}\left(1-q^{2^n}\right). \tag{31}$$

where m is the cardinality of the random set, r is the load factor, and $q = 1 - p(m,r)$,

Proof. Each iteration of the loop in Algorithm **??** has a collision test which is Bernoulli distributed with a probability of success $p(m,r)$, where success denotes no collision occurred. We are interested in the random length N of the bit string when this outcome occurs.

For the random variable N to realize a value n, every bit string smaller than length n must fail and a bit string of length n must succeed. There are $2^n - 1$ bit strings smaller than length n and each one fails with probability q, and so by the product rule the probability that they all fail is given by

$$q^{2^n-1}. \tag{a}$$

Given that every bit string smaller than length n fails, what is the probability that every bit string of length n fails? There are 2^n bit strings of length n, each of which fails with probability q as before and thus by the product rule the probability that they all fail is q^{2^n}, whose complement, the probability that not all bit strings of length n fail, is given by

$$1 - q^{2^n}. \tag{b}$$

By the product rule, the probability that every bit string smaller than length n fails and a bit string of length n succeeds is given by the product of (**??**) and (**??**),

$$q^{2^n-1}\left(1-q^{2^n}\right). \tag{c}$$

For (**??**) to be a probability mass function, two conditions must be met. First, its range must be a subset of $[0,1]$. Second, the summation over its domain must be 1.

The first case is trivially shown by the observation that q is a positive number between 0 and 1 and therefore any non-negative power of q is positive number between 0 and 1.

The second case is shown by calculating the infinite series

$$S = \sum_{n=0}^{\infty} q^{2^n-1}\left(1-q^{2^n}\right) \tag{d}$$

$$= \sum_{n=0}^{\infty} q^{2^n-1} - q^{2^{n+1}-1}. \tag{e}$$

Explicitly evaluating this series for the first 4 terms reveals a telescoping sum given by

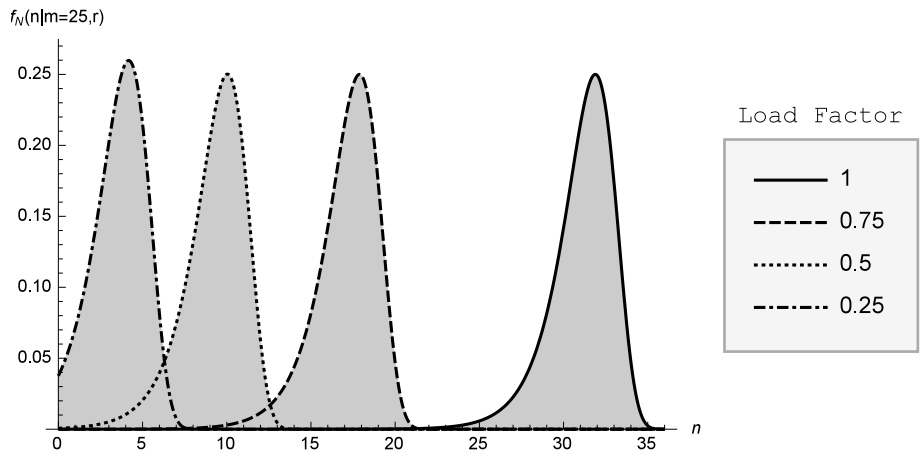
$$S = (1-q) + (q-q^3) + (q^3-q^7) + (q^7-q^{15}) + \cdots, \tag{f}$$

where everything cancels except 1. □

In Figure **??**, we plot the probability mass function of N conditioned on $m = 25$ and several different load factors. We see that the probability mass is peaked and unimodal and tends to nearly zero everywhere except over a concentrated interval around its expected value.

The expected size was already computed, but the probability mass function contains everything there is to know about the distribution of N, not just the *expected* value. However, for illustration, we show how the expected value may be computed.

Figure 2: Probability mass function of random bit length N



Theorem A.2. *The expected bit length of N conditioned on random sets of cardinality m and a load factor r is given by*

$$\mathbb{E}[N] = \sum_{j=1}^{\infty} q^{2^j - 1}, \tag{32}$$

where $q = 1 - p(m, r)$.

Proof. The expectation of N is given by

$$\sum_{j=0}^{\infty} j q^{2^j - 1} \left(1 - q^{2^j}\right) \tag{a}$$

$$= \sum_{j=0}^{\infty} j \left(q^{2^j - 1} - q^{2^{j+1} - 1}\right). \tag{b}$$

Explicitly evaluating this series for the first 4 terms reveals a converging sum given by

$$0 + (q - q^3) + 2(q^3 - q^7) + 3(q^7 - q^{15}) + \cdots \tag{c}$$

$$= q + q^3 + q^7 + q^{15} + \cdots. \tag{d}$$

□

When we numerically evaluate (**??**) for various m and r, the results are in agreement with (**??**).